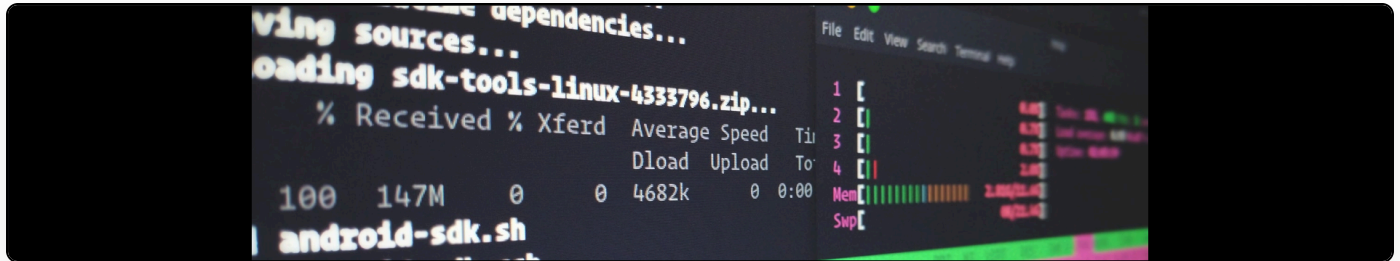


# Resizing image uploads in Django

Cite as: Martin Paul Eve, "Resizing image uploads in Django", *eve.gd: Martin Paul Eve*, April 30, 2022, <https://doi.org/10.59348/jrw3f-br877>.

---



It should be an easy task to resize image uploads in Django, but it turns out to be a bit more complicated than one would hope. Here are my findings.

Assume that you have defined a field in a model like this:

```
organization_logo = models.ImageField(  
    verbose_name='Associated image (145 x 145)',  
    upload_to=profile_images_upload_path,  
    blank=True,  
    null=True)
```

When you have a ModelForm associated with this, it will allow your users to upload an image. All well and good.

If you want to resize it, the easiest way is, surely, in the ModelForm, to override “def clean\_organization\_logo(self)” with some code that does the resize.

```
def clean_organization_logo(self):  
    img = self.cleaned_data.get('organization_logo')  
  
    return utils.resize_image(145, 145, img)
```

OK, so what does that function need to do?

Here's what I ended up with:

```
import _io
import io
from PIL import Image
from resizeimage import resizeimage

def resize_image(width, height, image_upload):
    # I don't know why this needs the type check, but sometimes
    # img.file is an image byte array and sometimes it's a
    # temporary file wrapper. This function handles both of those
    # eventualities

    if not image_upload:
        return image_upload

    with Image.open(image_upload.file) as image:
        cover = resizeimage.resize_cover(image, [width, height])

        if type(image_upload.file) is _io.BytesIO:
            img_byte_arr = io.BytesIO()
            cover.save(img_byte_arr, image.format)
            image_upload.file = img_byte_arr
        else:
            cover.save(image_upload.file.name, image.format)

    return image_upload
```

I used the [resize-image](#) module to do the actual work of resizing, but the gotcha is in what Django gives you. At different times, it gave me a `img.file` object that was different. Sometimes it would come back as an `_io.BytesIO` and sometimes as a `tempfile._TemporaryFileWrapper`. I have no idea why. In any case, this code seems to be working in all cases now.